

```

"""Tests for the Alert and AlertRule classes in src.alerts.

Author: Godspower Istifanus
"""

from datetime import datetime

import pytest

from src.alerts import Alert, AlertRule
from src.readings import SensorReading

@pytest.fixture
def timestamp() -> datetime:
    """Provide a fixed timestamp for deterministic test data."""
    return datetime(2024, 1, 1, 12, 0, 0)

def make_reading(
    sensor_id: str, value: float, unit: str = "C", timestamp: datetime = None
) -> SensorReading:
    """Build a SensorReading with sensible defaults for tests."""
    if timestamp is None:
        timestamp = datetime(2024, 1, 1, 12, 0, 0)
    return SensorReading(
        sensor_id=sensor_id, value=value, unit=unit, timestamp=timestamp
    )

class TestAlertSeverityValidation:
    """Tests covering Alert's severity validation in the constructor."""

    @pytest.mark.parametrize(
        "severity", ["LOW", "MEDIUM", "HIGH", "CRITICAL"]
    )
    def test_valid_severity_accepted(self, severity, timestamp):
        """Each valid severity level should construct without error."""
        alert = Alert(
            sensor_id="temp-1",
            severity=severity,
            message="test message",
            timestamp=timestamp,
        )

```

```

        assert alert.severity == severity

@pytest.mark.parametrize(
    "bad_severity", ["low", "URGENT", "", "Critical", "INFO", None]
)
def test_invalid_severity_raises_value_error(
    self, bad_severity, timestamp
):
    """Any severity outside the allowed set should raise ValueError."""
    with pytest.raises(ValueError):
        Alert(
            sensor_id="temp-1",
            severity=bad_severity,
            message="test message",
            timestamp=timestamp,
        )

class TestAlertOrdering:
    """Tests covering Alert.__lt__ ordering by severity."""

    def test_sorting_mixed_severity_alerts(self, timestamp):
        """Sorting a list of mixed-severity alerts should order them from
        least to most urgent (LOW < MEDIUM < HIGH < CRITICAL)."""
        critical = Alert("s1", "CRITICAL", "crit msg", timestamp)
        low = Alert("s2", "LOW", "low msg", timestamp)
        high = Alert("s3", "HIGH", "high msg", timestamp)
        medium = Alert("s4", "MEDIUM", "medium msg", timestamp)

        alerts = [critical, low, high, medium]
        alerts.sort()

        assert [a.severity for a in alerts] == [
            "LOW",
            "MEDIUM",
            "HIGH",
            "CRITICAL",
        ]

    def test_pairwise_comparison(self, timestamp):
        """Directly compare pairs of alerts to confirm rank ordering."""
        low = Alert("s1", "LOW", "msg", timestamp)
        critical = Alert("s2", "CRITICAL", "msg", timestamp)
        assert low < critical
        assert not critical < low

```

```

class TestAlertRuleCheckInRange:
    """Tests covering AlertRule.check() when readings are within range."""

    def test_returns_none_when_in_range(self, timestamp):
        """A reading within [min_threshold, max_threshold] yields no alert."""
        rule = AlertRule(
            sensor_id="temp-1", min_threshold=10.0, max_threshold=20.0
        )
        reading = make_reading("temp-1", 15.0, timestamp=timestamp)
        assert rule.check(reading) is None

    def test_returns_none_at_inclusive_boundary(self, timestamp):
        """Values exactly at min or max threshold are considered in range."""
        rule = AlertRule(
            sensor_id="temp-1", min_threshold=10.0, max_threshold=20.0
        )
        in_reading = make_reading("temp-1", 10.0, timestamp=timestamp)
        max_reading = make_reading("temp-1", 20.0, timestamp=timestamp)
        assert rule.check(in_reading) is None
        assert rule.check(max_reading) is None


class TestAlertRuleCheckSeverityTiers:
    """Tests covering severity tier selection at MEDIUM/HIGH/CRITICAL
    boundary values, breaching the max_threshold side of the range."""

    @pytest.fixture
    def rule(self):
        """A rule with max_threshold=100 to make percentage math simple."""
        return AlertRule(
            sensor_id="press-1", min_threshold=0.0, max_threshold=100.0
        )

    def test_medium_severity_within_10_percent_over(self, rule, timestamp):
        """A value 10% over the max threshold should be MEDIUM."""
        reading = make_reading("press-1", 110.0, timestamp=timestamp)
        alert = rule.check(reading)
        assert alert is not None
        assert alert.severity == "MEDIUM"

    def test_high_severity_between_10_and_25_percent_over(
        self, rule, timestamp
    ):
        """A value 25% over the max threshold should be HIGH (boundary)."""
        reading = make_reading("press-1", 125.0, timestamp=timestamp)
        alert = rule.check(reading)
        assert alert is not None

```

```

        assert alert.severity == "HIGH"

def test_critical_severity_more_than_25_percent_over(
    self, rule, timestamp
):
    """A value more than 25% over the max threshold should be CRITICAL."""
    reading = make_reading("press-1", 126.0, timestamp=timestamp)
    alert = rule.check(reading)
    assert alert is not None
    assert alert.severity == "CRITICAL"

def test_severity_tiers_below_min_threshold(self, timestamp):
    """Severity tiering should also work for breaches below the
    min_threshold side of the range."""
    rule = AlertRule(
        sensor_id="press-1", min_threshold=100.0, max_threshold=200.0
    )

    medium = rule.check(make_reading("press-1", 90.0, timestamp=timestamp))
    assert medium.severity == "MEDIUM" # 10% under

    high = rule.check(make_reading("press-1", 76.0, timestamp=timestamp))
    assert high.severity == "HIGH" # 24% under

    critical_reading = make_reading(
        "press-1", 50.0, timestamp=timestamp
    )
    critical = rule.check(critical_reading)
    assert critical.severity == "CRITICAL" # 50% under

class TestAlertRuleNonMatchingSensor:
    """Tests covering AlertRule.check() for readings from other sensors."""

    def test_returns_none_for_non_matching_sensor_id(self, timestamp):
        """A reading from a different sensor should be ignored entirely,
        even if its value would otherwise breach the threshold."""
        rule = AlertRule(
            sensor_id="temp-1", min_threshold=10.0, max_threshold=20.0
        )
        reading = make_reading("temp-2", 1000.0, timestamp=timestamp)
        assert rule.check(reading) is None

```